STAGE: STEMMED ACCOMPANIMENT GENERATION THROUGH PREFIX-BASED CONDITIONING

Giorgio Strano[1,⋆] **Chiara Ballanti**[1,⋆] **Donato Crisostomi**[1]
Michele Mancusi[1] **Luca Cosmo**[2] **Emanuele Rodolà**[1]
[1]Sapienza University of Rome, [2]Ca' Foscari University of Venice
strano@di.uniroma1.it

ABSTRACT

Recent advances in generative models have made it possible to create high-quality, coherent music, with some systems delivering production-level output. Yet, most existing models focus solely on generating music from scratch, limiting their usefulness for musicians who want to integrate such models into a human, iterative composition workflow. In this paper we introduce STAGE, our *STemmed Accompaniment GEneration* model, fine-tuned from the text-to-music MusicGen model to generate single-stem instrumental accompaniments conditioned on a given mixture. Inspired by instruction-tuning methods for language models, we extend the transformer's embedding matrix with a *context token*, enabling the model to attend to a musical context through prefix-based conditioning. Compared to the baselines, STAGE yields accompaniments that exhibit stronger coherence with the input mixture, higher audio quality, and closer alignment with textual prompts. Moreover, by conditioning on a metronome-like track, our framework naturally supports tempo-constrained generation, achieving state-of-the-art alignment with the target rhythmic structure–all without requiring any additional tempo-specific module. As a result, STAGE offers a practical, versatile tool for interactive music creation that can be readily adopted by musicians in real-world workflows.

github.com/giorgioskij/stage
giorgioskij.github.io/stage-demo

1. INTRODUCTION

Generative AI has recently transformed music composition, with large-scale models now able to produce long-form, high-quality, and stylistically consistent music. Models such as MusicGen [1], MusicLM [2], and JukeBox [3] have demonstrated that transformers trained on tokenized audio representations can generate music that rivals human compositions in coherence and production quality.

However, most of these models focus on generating music from scratch, even when they allow for conditional generation using melodies [1], chords [4–8], or text prompts. This limits their applicability in a natural music composition workflow, which is often structured in an iterative, layered fashion, gradually building compositions by adding or refining parts over time. To support this workflow, we focus on a human-centered, intuitive generation task: *adding a single new stem to an existing multi-stem mixture*, while also allowing for precise control over the tempo of the generated output.

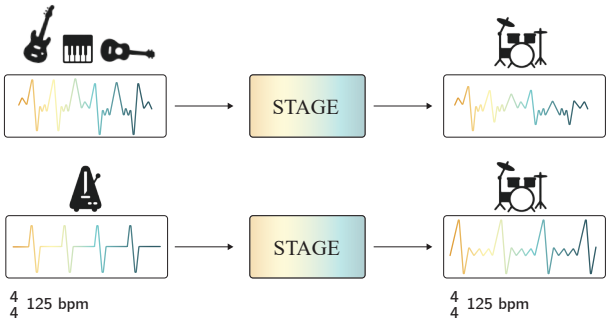


Figure 1. Outline of our proposed model. (top) STAGE takes a musical context as input and generates a single-stem accompaniment. (bottom) STAGE takes a metronome-like track and generates a stem that follows the desired rhythmic structure.

In this paper, we introduce STAGE, a single-instrument accompaniment generation model that can be conditioned on any audio context, be it a mixture or a simple click track, to generate a coherent and rhythmically aligned accompaniment (see Figure 1). We use a simple yet effective approach to fine-tune MusicGen [1] for stemmed accompaniment generation. Our method does not require retraining any additional context-encoders and relies on minimal data for fine-tuning. We leverage prefix-based conditioning, where the context is prepended to the model's input sequence, effectively serving as a prompt for the generation of the target stem. This approach draws inspiration from instruction tuning [9, 10] in language models, where prepending a task-specific instruction enables a pretrained model to specialize in new tasks with minimal modifications. In our case, we treat musical contexts as the "question" and the desired accompaniment as the "answer". This enables the model to learn a token-to-token correspondence between context and continuation, specializing it for accompaniment tasks.

We evaluate our model on musical coherence using the

⋆ denotes equal contribution.

COCOLA score [11], showing clear improvements over existing baselines, while maintaining high audio quality as measured by FAD [12] and KAD [13]. Furthermore, we show that our model supports tempo-constrained generation by simply conditioning on a metronome-like click track—without the need for tempo-specific modules or architectures.

Our contributions are three-fold:

- A prefix-based fine-tuning method for stemmed accompaniment generation.
- A lightweight, flexible approach to tempo conditioning through audio-based inputs.
- Extensive evaluation across musical coherence, rhythmic alignment, and audio fidelity, along with open-source code and model checkpoints.

2. RELATED WORK

Recent advances in generative modeling treat music as a language of discrete tokens, enabling long-context audio synthesis via transformer architectures. `Jukebox` [3] was an early example: it converted raw audio into a hierarchy of residual VQ-VAE tokens and used progressively deeper transformers to generate coherent extended sequences of music. Subsequent improvements in *neural audio codecs*, such as `SoundStream` [14] and `EnCodec` [15], inspired new designs. `MusicLM` [2] introduced a hierarchical two-stage approach that models separate streams of "semantic" and "acoustic" tokens, while `MusicGen` [1] showed that a single-stage transformer over `EnCodec` tokens can achieve excellent text-to-music quality. The simpler architecture of `MusicGen`, combined with its robust audio fidelity generations, make it a natural foundation for specialized tasks such as single stem accompaniment, which we explore in this paper.

2.1 Conditional generation

Although most music LMs focus on text prompting, some approaches provide more direct musical guidance or editing capabilities. Melody-conditioned models, such as the melody variant in [1], align the generation to a guiding pitch contour but typically produce an entire mix rather than an isolated stem. `MusicConGen` [4] further extends `MusicGen` by adding conditioning over chords and beat information, allowing explicit control of harmonic and rhythmic structures. Multiple other systems have been presented, especially using diffusion models, to condition music generation on a series of chords [5–8], on stylistic references [16], or even on a combination of text, style, and a reference drums track [17].

2.2 Music editing

Recent approaches to audio-domain editing include autoregressive models like `Instruct-MusicGen` [18], as well as diffusion-based systems such as `MSDM` [19] and `GMSDI` [20]. While `Instruct-MusicGen` struggles to achieve high-quality outputs, diffusion-based models, despite their flexibility, require significantly more computational resources and do not consistently support clean, single-stem generation.

2.3 Accompaniment generation

Similarly to `MSDM` and `GMSDI`, other diffusion-based systems aim to generate or edit partial arrangements but are closed-source or limited in scope. For instance, `Diff-A-Riff` [21] uses a multi-step diffusion process to refine an existing track with new musical elements; however, it is not openly released. `SA-ControlNet` [1] uses a fine-tuning of `Stable Audio Open` [22] with an added `ControlNet` module [23], also providing a form of stemmed accompaniment generation. `SingSong` [24] tackles vocal-to-instrumental accompaniment, taking a vocal track as input and generating a band-like backing. This approach is highly effective for vocals but remains highly specialized. More directly aligned with our objectives, `StemGen` [25] enables single-stem accompaniment generation via a non-autoregressive transformer. In parallel to our work, Meta AI introduced `MusicGen-Stem` [26], which supports a range of editing tasks, including mixture-conditioned accompaniment generation. However, neither model has released public code or checkpoints at the time of writing, and they only provide a handful of generated samples, making it impossible to perform a rigorous comparison. Additionally, both approaches involve training dedicated transformer models from scratch, in contrast to our lightweight fine-tuning strategy.

3. BACKGROUND

`MusicGen` [1] is a single-stage music generation model that operates over discrete audio tokens produced by an encoder–decoder neural codec. Specifically, the authors use `EnCodec` [15], which converts raw audio into several parallel streams of quantized tokens (known as codebooks). Whereas some prior works (e.g., `Jukebox` [3], `MusicLM` [2]) rely on multi-stage or hierarchical architectures that process one set of tokens to then upsample another, `MusicGen` proposes a simpler yet effective single-stage transformer language model that directly learns to generate all of these quantized tokens at once.

3.1 Architecture overview

The core of `MusicGen` is a GPT-like transformer decoder that is trained autoregressively over sequences of discrete audio tokens. The tokens come from a residual vector quantization scheme, where the raw waveform is first encoded into a low-frame-rate continuous representation, and then each frame is quantized by multiple "stacked" codebooks. Codebooks are organized hierarchically, and each codebook k_i contains incremental residual information w.r.t. the previous codebooks $k_j, i > j$. The number of codebooks (set to 4 in `MusicGen`) determines how many parallel tokens must be modeled at each time step.

[1] github.com/EmilianPostolache/stable-audio-controlnet

MusicGen is released in four versions, Small (∼400M params), Medium (∼1.5B params), Large (∼3B) and Melody (∼1.5B). The latter uses both text and a short audio clip (from which melody is extracted) as conditioning, with both sources prepended to the input. The other three rely only on text, provided via cross-attention. In this work, we use only the pre-trained MusicGen Small checkpoint.

3.2 Codebook Interleaving Patterns

One straightforward approach to convert the four parallel streams of tokens generated by EnCodec into a single stream is to flatten all codebooks, but this substantially lengthens the sequence. Conversely, predicting them in parallel underestimates cross-codebook dependencies. A practical compromise, used in MusicGen and shown in Figure 2, is the "delay" strategy which shifts tokens from the ith codebook by $i-1$ steps. This preserves inter-codebook context while requiring only one autoregressive step per frame. As a result, the delay pattern yields more efficient modeling than parallel prediction with minimal computational overhead.

4. METHOD

In this section, we present the key components of STAGE. In particular, we discuss how it extends MusicGen's architecture and training procedure to generate single stems from an input mixture or a metronome-like beat track. We refer to both these audio inputs as the "context". During inference, the user can pass either of those audio cues or even combine them into a single waveform, allowing to perform accompaniment generation with an even tighter control over the target's rhythmic structure.

4.1 Overview

Having chosen a target instrument I, STAGE is trained to produce a stem S of that instrument, taking as input:

- An audio context, which can contain a generic audio mixture M (without our target instrument I), **or** a beat track B (a metronome-like pulse sequence);
- an optional text prompt T, describing the desired style and mood of the generation.

The model aims to generate audio that matches the mixture's key, style, harmony, and rhythm, enabling it to serve as a musical accompaniment. If no initial mixture is available, STAGE can instead take a metronome track (B) (a simple beat marking the tempo) as input, allowing it to generate the first stem of a composition. In practice, we find that overlaying the mixture M and beat track B into a single audio file at inference time allows STAGE to condition on both, preserving coherence with the mixture while achieving tighter rhythmic alignment with the beats (see 5.4 for results).

We train separate STAGE models for each target instrument, specifically *drums* and *bass*. For the remaining, less-represented instruments, the inherent intra-instrument variability would require significantly more data than what is available in MoisesDB.

4.2 Context token for prefix-based conditioning

Our starting point is MusicGen-Small, a lightweight variant of MusicGen. To enable conditioning, we add a single *context token* to the transformer's embedding matrix, allowing extra audio tokens to be prepended to the input sequence (Figure 2). These tokens come from either the mixture M or the beat track B, encoded via EnCodec [15]. This forms a prefix-based conditioning setup: once the model ''sees" the context tokens, it autoregressively generates the new stem.

4.3 Fine-tuning procedure

We train on the open-source, multi-stem dataset MoisesDB [27], which contains 240 stem-separated songs. Given a target instrument I to be generated, we create input data for STAGE using the following strategies:

- **Form the context.** For each track we either (a) mix a random subset of stems (excluding instrument I) to create M, or (b) replace the mixture with a metronome track B at the known tempo of M. We use each of the two strategies with equal probability.
- **Context length.** We randomize the context length in the range of 5 to 10 seconds. Hence, the model can learn to generate samples longer than the actual context window.
- **Data augmentation.** We apply speed transposition (in the range [0.8, 1.2]) and pitch transposition (in the range [-4, +4 semitones]) with probability 0.5 to both context and target.

The above procedure produces <context, target stem> pairs, which we use for fine-tuning. To allow the newly introduced *context token* to adapt to the pretrained model, we first train only its embedding for 200 steps at a learning rate of 1e-4, keeping all other weights frozen. This warm-up phase helps the token learn a meaningful interface with the rest of the model. We then unfreeze the remaining weights and gradually ramp up their learning rate from 0 to 1e-5, while annealing the context token's rate from 1e-4 to 1e-5. Training converges in about 1,000 steps using batches of eight 10-second samples, finishing in under a day on a single NVIDIA RTX 3090 GPU.

4.4 Inference

At inference time, we perform the following steps:

- Tokenize the context via EnCodec.
- Pass the encoded tokens to STAGE, followed by the context token.

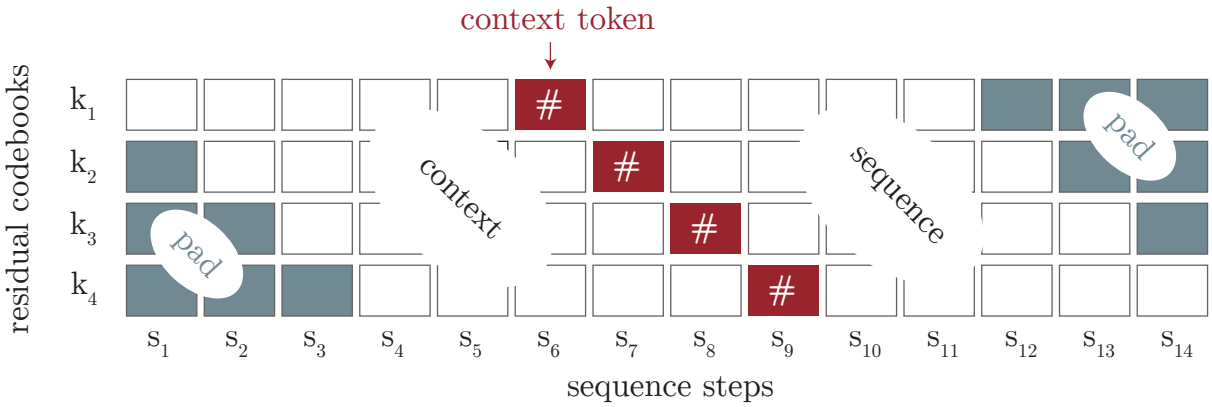


Figure 2. Illustration of the *delay* pattern used by MusicGen, and how the context token is placed to separate the audio context from the input sequence of the transformer.

- Autoregressively decode the new stem's tokens.
- Reconstruct the generated stem via `EnCodec`'s decoder.

This lightweight prefix-based framework is easy to integrate into a musician's workflow by simply providing an audio snippet of an existing track (or a metronome pulse in its place) and letting `STAGE` generate a single, coherent accompaniment stem.

5. EXPERIMENTS AND RESULTS

We now present the experimental setup and evaluation metrics used to assess the performance of `STAGE`. We focus our evaluation on two separate tasks:

- *Beat Alignment*, given a beat as conditioning;
- *Accompaniment Generation*, given a generic mixture of stems as conditioning.

For each of these tasks, we measure against state-of-the-art open-source models on comparable tasks.

5.1 Beat alignment

To measure how precisely `STAGE` follows the given beat, we provide the model with a raw pulse track spanning the full duration of the sample to be generated. We then evaluate the F1 score using the `mireval` [2] library, matching the detected beats in the output audio to the reference beat track supplied as input. For beat detection, we employ Beat-This [28], a state-of-the-art algorithm for beat tracking.

We benchmark `STAGE` against `MusicConGen` [4], a fine-tuned `MusicGen` variant that can be conditioned on tempo, and optionally on chord sequences.

Following the setup of `MusicConGen`, we evaluate our generations by conditioning on beat tracks extracted from MusDB [29] mixtures using the Beat-This algorithm. However, we find that this process introduces noise, as the beat tracker is imperfect and often detects irregular or inconsistent beat patterns across samples. This is not in line with a realistic scenario, in which a musician supplies the model with a perfectly regular beat grid to follow. For a more realistic setting, we also test our model on a uniform distribution of BPMs in the interval [100, 180].

Table 1 shows that the model trained on drums significantly outperforms both `MusicConGen` and our bass-trained variant. This supports the intuitive notion that training on drums gives the model a stronger sense of rhythm, leading to better alignment. Additionally, beat extraction is less accurate on bass-only tracks (like those generated by `STAGE-bass`) which can contribute to higher measured alignment error.

Model	Dataset	F1 ↑	FAD-VGGish ↓	FAD-Clap ↓
STAGE-drums	MusDB	66.88	1.40	0.23
	Uniform BPM	71.57	2.05	0.24
STAGE-bass	MusDB	40.93	5.59	0.39
	Uniform BPM	45.17	4.26	0.52
MusiConGen-Tempo	MusDB	61.37	1.95	–

Table 1. Comparison of audio quality (FAD) and rhythmic alignment (F1) of `STAGE-drums` vs. `MusicConGen` (with Rhythm-only conditioning). The rhythm conditioning was extracted with Beat-This from the MusDB dataset. For our model, we also test on 160 samples from a uniform distribution of BPMs in the range [100, 180]. Following [4], FAD is computed with MusDB as reference.

5.2 Accompaniment coherence

To assess how well our model adds a new instrument stem to an existing mixture, we evaluate on the test set from the MoisesDB dataset [27]. For each track, we remove the target stem (drums or bass) and then ask the model to regenerate that instrument while keeping the remaining parts unchanged. We measure several metrics that target both objective audio quality and semantic coherence with the context:

[2] github.com/mir-evaluation/mir_eval

Target stem	Model	COCOLA ↑	FAD-VGGish ↓	FAD-Clap ↓	KAD-VGGish ↓	KAD-Clap ↓	Rhythmic Alignment (F1) ↑
Drums	STAGE	**61.02**	**1.05**	**0.17**	**3.00**	**9.86**	**52.63**
	Instruct-MusicGen	48.24	16.44	1.34	55.25	80.79	0.20
	GMSDI	45.25	15.57	1.22	57.39	54.75	25.98
	SA-ControlNet	57.46	2.75	0.39	9.28	11.75	38.70
Bass	STAGE	**60.20**	2.97	0.37	13.45	**15.40**	40.94
	Instruct-MusicGen	53.69	12.85	1.28	47.41	73.92	0.19
	GMSDI	43.29	14.36	1.28	44.31	49.55	24.34
	SA-ControlNet	59.63	**2.22**	**0.31**	**6.69**	15.98	**47.17**

Table 2. Performance comparison on the accompaniment generation task using the MoisesDB test set, with *bass* and *drums* as target stems.

- COCOLA score [11] captures harmonic and percussive coherence between the context and the newly generated stem;
- FAD-vggish and FAD-clap [12] assess perceptual quality by comparing the distribution of embeddings (extracted using VGGish [30] or CLAP [31]) between generated and reference audio. Since FAD measures distributional distance, we use stems from MoisesDB matching the target instrument to define the reference.
- KAD-vggish and KAD-clap [13]. A newly released metric that computes the distance between distributions of embeddings in a higher-order abstract space, using the kernel trick. It has similar properties to the commonly used FAD metric.
- Rhythmic Alignment (F1). We also compute the F1-score between beats extracted from the context, and beats extracted from the generated stem, to asses the rhythmic alignment and coherence between context and accompaniment.

The scores for GMSDI and SA-ControlNet are computed on samples provided by the authors, while Instruct-MusicGen was run locally with the public inference code [3] . We were not able to compare with StemGen [25], Diff-A-Riff [21], and MusicGen-Stem [26] since their code is not publicly available.

As shown in Table 2, our proposed model outperforms all baselines in semantic coherence, audio quality, and rhythmic alignment when generating *drums*, and performs on par with SA-ControlNet for *bass*. Overall, we observe that STAGE performs slightly worse on bass across all metrics. We attribute this to two main factors:

a) Greater variability in the distribution of bass tracks within the MoisesDB dataset, which includes both electric and synth bass with distinct timbral characteristics.

b) The inherently more complex nature of bass generation, which–unlike drums–requires modeling both rhythmic and harmonic information from the context, resulting in a more challenging prediction task.

[3] github.com/ldzhangyx/instruct-MusicGen

5.3 Ablation study on rhythm conditioning

To evaluate the impact of training on <metronome, target> pairs alongside <mixture, target> pairs, we conduct an ablation study (Table 3). We fine-tune a version of STAGE-drums without metronome tracks, exposing it only to <mixture, target> pairs. As expected, the model fails to generate tempo-aligned outputs when conditioned on a metronome at inference. More notably, its performance *also degrades on accompaniment generation* (the very task it was trained for) highlighting the broader benefit of including metronome conditioning at training.

The clear gap in both COCOLA scores (which capture rhythmic and harmonic coherence) and F1 scores for rhythmic alignment (Table 3) indicates that including <metronome, target> pairs during fine-tuning not only enables tempo-constrained generation, but also enhances the model's overall ability to perceive and reproduce rhythmic structure in standard accompaniment generation.

5.4 Combining mixture and metronome for improved alignment

As mentioned in 4.1, we verify that, at inference time, we can condition the model on a combination of mixture and metronome, by simply summing the waveforms. Even though STAGE has never seen such contexts during training, it is able to generalize to these conditionings and provide accompaniment generation with even greater control over the rhythmic alignment. We measure this effect on the same accompaniment generation task by comparing beat alignment (F1 score) when either conditioning on M alone vs. on $M + B$. Results, shown in Figure 3 confirm that an explicit beat track helps tighten alignment while preserving coherence with the mixture.

6. DISCUSSION

Below we discuss a few more general takeaways from this research, and share our opinions about research on large generative transformer models.

Target stem	Model	COCOLA ↑	FAD-VGGish ↓	FAD-Clap ↓	KAD-VGGish ↓	KAD-Clap ↓	Rhythmic Alignment (F1) ↑
Drums	STAGE	**61.02**	**1.05**	0.17	**3.00**	9.86	**52.63**
	STAGE-abl.	59.28	1.16	0.17	4.37	**9.59**	48.05

Table 3. Ablation results: the two models are exactly the same, but STAGE-abl never sees <metronome, target> pairs during fine-tuning. Models are tested on accompaniment generation, generating drums given a mixture of stems extracted from the test set of MoisesDB. We observe that the model which observed conditioning with metronome tracks is able to better align to the rhythmic structure of the mixture, even on the normal accompaniment generation task.

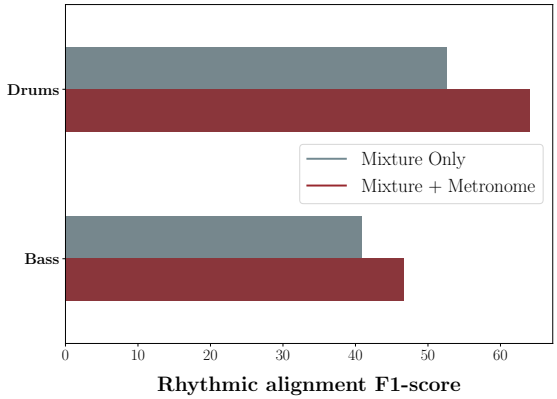


Figure 3. Comparison of rhythmic alignment when passing only a mixture as conditioning vs. the combination of the same mixture with a metronome track. For STAGE-drums, the F1 alignment improves from 52.6 to 64.0, and for STAGE-bass from 40.9 to 46.8.

6.1 Parameter-efficiency of single-stem fine-tuning

As shown in Table 4, STAGE uses about one-tenth the trainable parameters of comparably performing systems, yet outperforms them across multiple tasks. By training on a single stem with simple prefix-based conditioning, it aligns closely with the audio context without extra encoders or modules. Fine-tuning a general model like MusicGen on a single stem focuses predictive power on a simpler waveform distribution, making it **a highly parameter-efficient technique**.

These experiments suggest that stacking multiple STAGE instances, one per instrument, can match or exceed general models' performance at a fraction of the parameters. This aligns with a broader trend: recent research on large language models shows that *assigning subsets of parameters to specific subtasks* is often more efficient than scaling monolithic models. DeepSeek V3 [32] exemplifies this with its use of Mixture of Experts, although in a different context. We believe that, given the exponentially increasing cost of very large models, attention to more parameter-efficient architectures should not be spared.

Model	# Params
STAGE	∼0.4B
Instruct-Musicgen	∼4.7B
GMSDI	∼0.8B
SA ControlNet	∼3.8B

Table 4. Considered models' trainable parameters count.

6.2 Cross-attention in local vs. global conditioning

Before pivoting towards prefix-based conditioning, we extensively tested cross-attention for injecting musical context into the model. While fine-tuned models captured style, mood, and harmony, they struggled with precise rhythmic alignment, even with positional embeddings designed for local dependencies. Moving the conditioning source into the input stream, processed via self-attention, resolved this issue entirely, enabling accurate alignment between conditioning and output. This behavior has rarely been documented, with [33] noting that *cross-attention outputs converge to a fixed point in the first few steps*, splitting the process into semantic planning (via cross-attention), and subsequent image generation.

In [34], there are hints of a similar intuition. We conclude that the precise behavior of cross-attention in traditional transformer decoders, and its difference in effectiveness when handling global (e.g.: the global description of an image, the meaning of a text to translate) vs. highly localized conditioning (e.g.: the exact positions of the beats to follow to in a musical piece) is way underexplored, and still deserves attention for future research.

7. CONCLUSION

We introduced STAGE, a parameter-efficient single-instrument approach for accompaniment generation that extends MusicGen with a simple, prefix-based conditioning mechanism. By prepending an audio context to the model's input, STAGE effectively learns the relationship between context and accompaniment, delivering competitive or superior performance in audio fidelity, contextual coherence, and beat alignment compared to larger baseline systems, despite its compact size and minimal fine-tuning effort. We explored how enabling the model to be conditioned on either a full mixture or a simple metronome track, not only enables it to perform strictly temporally-controlled generation, but also improves its rhythmic alignment in the more general accompaniment generation task. Future work may explore extending STAGE to additional instruments and refining its capacity for an even more granular degree of control, further expanding its applicability in real-world music production workflows.

8. ACKNOWLEDGMENTS

We acknowledge support from Sapienza University of Rome through the Seed of ERC grant "MINT.AI", cup B83C25001040001. L.C. is supported by the PRIN 2022 project n. 2022AL45R2 (EYE-FI.AI, CUP H53D2300350-0001), funded by the European Union – NextGenerationEU – PNRR – M4C2, Investment 1.1. We thank Emilian Postolache for useful discussions during the early stages of this work.

9. REFERENCES

[1] J. Copet, F. Kreuk, I. Gat, T. Remez, D. Kant, G. Synnaeve, Y. Adi, and A. Défossez, "Simple and controllable music generation," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

[2] A. Agostinelli, T. I. Denk, Z. Borsos, J. Engel, M. Verzetti, A. Caillon, Q. Huang, A. Jansen, A. Roberts, M. Tagliasacchi *et al.*, "Musiclm: Generating music from text," *arXiv preprint arXiv:2301.11325*, 2023.

[3] P. Dhariwal, H. Jun, C. Payne, J. W. Kim, A. Radford, and I. Sutskever, "Jukebox: A generative model for music," 2020. [Online]. Available: https://arxiv.org/abs/2005.00341

[4] Y.-H. Lan, W.-Y. Hsiao, H.-C. Cheng, and Y.-H. Yang, "Musicongen: Rhythm and chord control for transformer-based text-to-music generation."

[5] S. Gao, S. Lei, F. Zhuo, H. Liu, F. Liu, B. Tang, Q. Huang, S. Kang, and Z. Wu, "An end-to-end approach for chord-conditioned song generation," in *Proc. Interspeech 2024*, 2024, pp. 1890–1894.

[6] K. Choi, J. Park, W. Heo, S. Jeon, and J. Park, "Chord conditioned melody generation with transformer based decoders," *IEEE Access*, vol. 9, pp. 42 071–42 080, 2021.

[7] S. Li and Y. Sung, "Melodydiffusion: Chord-conditioned melody generation using a transformer-based diffusion model," *Mathematics*, vol. 11, no. 8, p. 1915, 2023.

[8] J. Jung, A. Jansson, and D. Jeong, "Musicgen-chord: Advancing music generation through chord progressions and interactive web-ui," *arXiv preprint arXiv:2412.00325*, 2024.

[9] J. Wei, M. Bosma, V. Zhao, K. Guu, A. W. Yu, B. Lester, N. Du, A. M. Dai, and Q. V. Le, "Finetuned language models are zero-shot learners," in *International Conference on Learning Representations*, 2022. [Online]. Available: https://openreview.net/forum?id=gEZrGCozdqR

[10] S. Zhang, L. Dong, X. Li, S. Zhang, X. Sun, S. Wang, J. Li, R. Hu, T. Zhang, F. Wu, and G. Wang, "Instruction tuning for large language models: A survey," 2024. [Online]. Available: https://arxiv.org/abs/2308.10792

[11] R. Ciranni, G. Mariani, M. Mancusi, E. Postolache, G. Fabbro, E. Rodolà, and L. Cosmo, "Cocola: Coherence-oriented contrastive learning of musical audio representations," in *ICASSP 2025-2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2025, pp. 1–5.

[12] K. Kilgour, M. Zuluaga, D. Roblek, and M. Sharifi, "Fréchet audio distance: A reference-free metric for evaluating music enhancement algorithms," 2019.

[13] Y. Chung, P. Eu, J. Lee, K. Choi, J. Nam, and B. S. Chon, "Kad: No more fad! an effective and efficient evaluation metric for audio generation," *arXiv preprint arXiv:2502.15602*, 2025.

[14] N. Zeghidour, A. Luebs, A. Omran, J. Skoglund, and M. Tagliasacchi, "Soundstream: An end-to-end neural audio codec," *IEEE/ACM Trans. Audio, Speech and Lang. Proc.*, vol. 30, p. 495–507, Nov. 2021. [Online]. Available: https://doi.org/10.1109/TASLP.2021.3129994

[15] A. Défossez, J. Copet, G. Synnaeve, and Y. Adi, "High fidelity neural audio compression," 2022. [Online]. Available: https://arxiv.org/abs/2210.13438

[16] S. Rouard, Y. Adi, J. Copet, A. Roebel, and A. Défossez, "Audio conditioning for music generation via discrete bottleneck features," in *ISMIR 2024*, 2024.

[17] O. Tal, A. Ziv, I. Gat, F. Kreuk, and Y. Adi, "Joint audio and symbolic conditioning for temporally controlled text-to-music generation," 2024.

[18] Y. Zhang, Y. Ikemiya, W. Choi, N. Murata, M. A. M. Ramírez, L. Lin, G. Xia, W.-H. Liao, Y. Mitsufuji, and S. Dixon, "Instruct-musicgen: Unlocking text-to-music editing for music language models via instruction tuning," *CoRR*, 2024.

[19] G. Mariani, I. Tallini, E. Postolache, M. Mancusi, L. Cosmo, E. Rodola *et al.*, "Multi-source diffusion models for simultaneous music generation and separation," in *12th International Conference on Learning Representations, ICLR 2024*. International Conference on Learning Representations, ICLR, 2024.

[20] E. Postolache, G. Mariani, L. Cosmo, E. Benetos, and E. Rodolà, "Generalized multi-source inference for text conditioned music diffusion models," in *ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2024, pp. 6980–6984.

[21] J. Nistal, M. Pasini, C. Aouameur, S. Lattner, and M. Grachten, "Diff-a-riff: Musical accompaniment co-creation via latent diffusion models."

[22] Z. Evans, J. D. Parker, C. Carr, Z. Zukowski, J. Taylor, and J. Pons, "Stable audio open," in *ICASSP 2025-2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2025, pp. 1–5.

[23] L. Zhang, A. Rao, and M. Agrawala, "Adding conditional control to text-to-image diffusion models," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, October 2023, pp. 3836–3847.

[24] C. Donahue, A. Caillon, A. Roberts, E. Manilow, P. Esling, A. Agostinelli, M. Verzetti, I. Simon, O. Pietquin, N. Zeghidour *et al.*, "Singsong: Generating musical accompaniments from singing," in *International Conference on Machine Learning ICML 2023*, 2023.

[25] J. D. Parker, J. Spijkervet, K. Kosta, F. Yesiler, B. Kuznetsov, J.-C. Wang, M. Avent, J. Chen, and D. Le, "Stemgen: A music generation model that listens," in *ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2024, pp. 1116–1120.

[26] S. Rouard, R. San Roman, Y. Adi, and A. Roebel, "Musicgen-stem: Multi-stem music generation and edition through autoregressive modeling," in *ICASSP 2025-2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2025, pp. 1–5.

[27] I. Pereira, F. Araújo, F. Korzeniowski, and R. Vogl, "Moisesdb: A dataset for source separation beyond 4-stems," *arXiv preprint arXiv:2307.15913*, 2023.

[28] F. Foscarin, J. Schlüter, and G. Widmer, "Beat this! accurate beat tracking without dbn postprocessing."

[29] Z. Rafii, A. Liutkus, F.-R. Stöter, S. I. Mimilakis, and R. Bittner, "The MUSDB18 corpus for music separation," Dec. 2017. [Online]. Available: https://doi.org/10.5281/zenodo.1117372

[30] S. Hershey, S. Chaudhuri, D. P. Ellis, J. F. Gemmeke, A. Jansen, R. C. Moore, M. Plakal, D. Platt, R. A. Saurous, B. Seybold *et al.*, "Cnn architectures for large-scale audio classification," in *2017 ieee international conference on acoustics, speech and signal processing (icassp)*. IEEE, 2017, pp. 131–135.

[31] B. Elizalde, S. Deshmukh, M. Al Ismail, and H. Wang, "Clap learning audio concepts from natural language supervision," in *ICASSP 2023-2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2023, pp. 1–5.

[32] A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan *et al.*, "Deepseek-v3 technical report," *arXiv preprint arXiv:2412.19437*, 2024.

[33] H. Liu, W. Zhang, J. Xie, F. Faccio, M. Xu, T. Xiang, M. Z. Shou, J.-M. Perez-Rua, and J. Schmidhuber, "Faster diffusion via temporal attention decomposition," 2025. [Online]. Available: https://arxiv.org/abs/2404.02747

[34] D. Liang, W. Longyue, W. Di, T. Dacheng, and T. Zhaopeng, "Context-aware cross-attention for non-autoregressive translation," *Proceedings of the 28th International Conference on Computational Linguistics*, pp. 4396–4402, 2020. [Online]. Available: https://cir.nii.ac.jp/crid/1360299149686305408